

Java Testing Basics - Cheat Sheet

Testing isn't optional in production code. The question is whether you write the tests intentionally or wait for production traffic to write them for you. JUnit 5 and Mockito are the default tools in modern Java. Once you're fluent in them, writing tests becomes a habit rather than a chore.

Why Tests Matter

Tests do four things:

- **Catch regressions.** Code that used to work, still works.
- **Document behavior.** Tests describe what the code is supposed to do.
- **Enable refactoring.** With tests in place, you can change implementation freely.
- **Force good design.** Hard-to-test code is usually badly designed code.

If code is painful to test, the code is usually the problem.

Types of Tests (The Test Pyramid)

Level	What it tests	Speed	Quantity
Unit	One class or function in isolation	Milliseconds	Lots
Integration	Two or more components together (DB, HTTP, etc.)	Seconds	Some
End-to-end	Full system through real APIs and UIs	Minutes	Few

The classic pyramid: many unit tests, fewer integration tests, very few e2e tests. Unit tests are fast and reliable. E2e tests are slow and flaky. You want most of your coverage at the bottom.

JUnit 5 Basics

A test class lives in `src/test/java` and looks like a regular Java class with `@Test` methods.

```

class CalculatorTest {

    @Test
    void addsTwoNumbers() {
        Calculator calc = new Calculator();
        int result = calc.add(2, 3);
        assertEquals(5, result);
    }
}

```

Test classes don't need to be `public`. Method names should describe the behavior being verified. Maven and Gradle test plugins discover anything matching `*Test` or `Test*`.

JUnit 5 is split into three pieces:

- **JUnit Platform:** the launcher and test engine API.
- **JUnit Jupiter:** the modern programming model (`@Test`, `@BeforeEach`, etc.).
- **JUnit Vintage:** runs JUnit 3 and 4 tests on the new platform.

For new projects, just use Jupiter.

Common Annotations

Annotation	What it does
<code>@Test</code>	Marks a test method
<code>@BeforeEach</code>	Runs before every test method
<code>@AfterEach</code>	Runs after every test method
<code>@BeforeAll</code>	Runs once before the entire test class (static)
<code>@AfterAll</code>	Runs once after the entire test class (static)
<code>@DisplayName("...")</code>	Sets a human-readable name shown in reports
<code>@Disabled("reason")</code>	Skips a test with a reason
<code>@Nested</code>	Groups related tests into a nested inner class
<code>@Tag("slow")</code>	Tags tests for selective running
<code>@Timeout(value, unit)</code>	Fails if the test runs longer than the timeout
<code>@RepeatedTest(n)</code>	Runs the test n times
<code>@ParameterizedTest</code>	Runs the test once per input set

Assertions

`org.junit.jupiter.api.Assertions.*` ships the standard set.

```
assertEquals(expected, actual);
assertEquals(expected, actual, "context message");
assertNotEquals(unexpected, actual);

assertTrue(value > 0);
assertFalse(list.isEmpty());

assertNull(obj);
assertNotNull(obj);

assertSame(expected, actual);    // reference equality
assertNotSame(unexpected, actual);

assertArrayEquals(new int[]{1, 2, 3}, result);
assertIterableEquals(expected, actual);

// Multiple assertions reported together
assertAll(
    () -> assertEquals("alice", user.getName()),
    () -> assertEquals(30, user.getAge()),
    () -> assertTrue(user.isActive())
);

// Exception assertions
assertThrows(IllegalArgumentException.class, () -> service.process(null));

// Float comparisons need a delta
assertEquals(1.0, computed, 0.0001);
```

For richer matchers, AssertJ is the most popular choice:

```
import static org.assertj.core.api.Assertions.assertThat;

assertThat(user.getName()).isEqualTo("alice");
assertThat(orders).hasSize(3).extracting(Order::status).containsOnly("paid");
assertThat(error).isInstanceOf(ValidationException.class)
    .hasMessageContaining("email");
```

AssertJ chains read like English. Worth picking up.

Test Lifecycle

```
class UserServiceTest {

    private UserService service;

    @BeforeAll
    static void setupClass() {
        // once before all tests in this class
    }

    @BeforeEach
    void setUp() {
        service = new UserService(); // fresh for each test
    }

    @Test
    void doesSomething() {
        // ...
    }

    @Test
    void doesSomethingElse() {
        // ...
    }

    @AfterEach
    void tearDown() {
        // cleanup after each test
    }

    @AfterAll
    static void teardownClass() {
        // once after all tests
    }
}
```

Each test method runs on a fresh instance by default. `@BeforeAll` and `@AfterAll` must be static unless you change the lifecycle with `@TestInstance(Lifecycle.PER_CLASS)`.

Parameterized Tests

Same test logic, multiple inputs.

```

@ParameterizedTest
@ValueSource(ints = {1, 2, 4, 8, 16})
void isPowerOfTwo(int n) {
    assertTrue(MathUtils.isPowerOfTwo(n));
}

@ParameterizedTest
@CsvSource({
    "alice, 30",
    "bob, 25",
    "charlie, 40"
})
void buildsUserFromCsv(String name, int age) {
    User u = new User(name, age);
    assertEquals(name, u.getName());
    assertEquals(age, u.getAge());
}

@ParameterizedTest
@MethodSource("userProvider")
void validatesUser(User user, boolean expected) {
    assertEquals(expected, validator.isValid(user));
}

static Stream<Arguments> userProvider() {
    return Stream.of(
        Arguments.of(new User("alice", 30), true),
        Arguments.of(new User("", 30), false),
        Arguments.of(new User("bob", -1), false)
    );
}

```

Sources available: `@ValueSource`, `@CsvSource`, `@CsvFileSource`, `@MethodSource`, `@EnumSource`.

Mocking with Mockito

Mockito replaces real collaborators with controlled stand-ins. Used everywhere in Java testing.

```

import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository repo;

    @Mock
    private EmailSender email;

    @InjectMocks
    private OrderService service;

    @Test
    void savesOrderAndSendsEmail() {
        // Arrange
        Order order = new Order("alice@example.com", 100.00);
        when(repo.save(any(Order.class))).thenReturn(order);

        // Act
        service.place(order);

        // Assert
        verify(repo).save(order);
        verify(email).send(eq("alice@example.com"), anyString());
    }
}

```

Common patterns:

```

// Stub a return value
when(repo.findById(42L)).thenReturn(Optional.of(user));

// Throw an exception
when(repo.save(any())).thenThrow(new RuntimeException("db down"));

// Verify call count
verify(repo, times(1)).save(any());
verify(repo, never()).delete(any());
verify(repo, atLeastOnce()).findById(anyLong());

// Argument matchers
verify(email).send(eq("alice"), contains("welcome"));

// Capture arguments for deeper inspection
ArgumentCaptor<Order> captor = ArgumentCaptor.forClass(Order.class);
verify(repo).save(captor.capture());
assertEquals("alice", captor.getValue().getCustomerName());

```

Rules:

- All arguments use matchers (`any()` , `eq()`) or all are raw values. Don't mix them.
- Avoid stubbing more than you need. Over-stubbing makes tests brittle.
- Static methods need `mockStatic(SomeClass.class)` (Mockito 3.4+).

Test Doubles

Five terms, often confused. Gerard Meszaros's definitions:

Type	What it does
Dummy	Placeholder, never used (<code>null</code> or an empty object)
Stub	Returns hardcoded values to drive the test
Spy	Real object that records calls for verification
Mock	Pre-programmed object that verifies interactions
Fake	Working implementation, but unsuitable for production (in-memory DB)

In Mockito, `@Mock` covers stubs and mocks. `@Spy` is the real-object spy. Fakes are usually hand-written (e.g., a `HashMap`-backed `Repository` for tests).

Arrange, Act, Assert

The structural pattern for almost every unit test.

```
@Test
void debitsAccountOnPurchase() {
    // Arrange
    Account account = new Account(100);
    Item item = new Item(30);

    // Act
    account.purchase(item);

    // Assert
    assertEquals(70, account.balance());
}
```

Three clear sections. Blank line between them, sometimes with a comment. Keeps tests readable and forces you to think about state, action, and outcome separately.

Test Naming

A good test name tells you what the code does and what's being verified, without making you read the body.

Bad:

- `test1()`
- `testGetUser()`
- `userTest()`

Good:

- `returnsUserWhenIdExists()`
- `throwsNotFoundWhenIdMissing()`
- `addsTwoNumbers_givenPositiveValues()`

Patterns that work:

- `methodName_state_expectedResult` (verbose, clear)
- `shouldXWhenY()` (BDD-style)
- Plain English: `debitsAccountOnPurchase()`

Pick a convention and stick with it across the codebase.

Best Practices

- **One assertion per test (mostly).** Or related assertions covering the same outcome.
 - **Test behavior, not implementation.** A refactor that keeps behavior the same shouldn't break tests.
 - **Keep tests independent.** A test should pass when run alone or in any order.
 - **No shared mutable state.** Static fields, singletons, or filesystem touches between tests cause flakiness.
 - **Mock external dependencies.** Don't mock the class you're actually testing.
 - **Prefer real objects when fast.** Mock only when the real thing is slow, unreliable, or hard to set up.
 - **Use AAA structure.** Arrange, Act, Assert. Skip it only with good reason.
 - **Test the boundaries.** Null inputs, empty collections, max/min values, off-by-one cases.
 - **Keep tests fast.** Slow tests get skipped, even by you.
-

Common Gotchas

- **Tests that pass alone but fail in a suite.** Shared static state. Reset it in `@BeforeEach` or remove it entirely.
 - **Tests that fail randomly.** Time-of-day dependencies, random ordering, network calls. Find the source of nondeterminism.
 - **Mockito `UnnecessaryStubbingException`** . You stubbed a method the code never called. Either remove the stub or check why the path isn't executing.
 - **`InvalidUseOfMatchersException`** . Mixed matchers and raw values in the same call. Use matchers for everything or for nothing.
 - **Mocking value objects or DTOs.** Don't. Just construct them. Mocks are for collaborators that do something, not for data containers.
 - **Testing private methods.** If a private method needs its own tests, it probably belongs in a separate class. Test through the public API.
 - **Tests that touch the network.** Slow and dependent on external services. Mock the HTTP client or use WireMock.
 - **Assertions on order without ordering guarantees.** `Set` and `HashMap` have no order. Use ordered collections or `containsExactlyInAnyOrder` .
 - **Tests larger than the code they test.** A sign that the production code is doing too much or the test is asserting too much.
-

Quick Reference

Concept	Key fact
JUnit 5	The standard test framework (Jupiter is the module)
@Test	Marks a test method
@BeforeEach / @AfterEach	Per-test setup and teardown
@BeforeAll / @AfterAll	Per-class setup and teardown (static)
Assertions	assertEquals , assertTrue , assertThrows , assertAll
AssertJ	Fluent assertions, reads like English
Mockito @Mock	Replace a collaborator with a stand-in
Mockito @InjectMocks	Auto-wire mocks into the class under test
when(...).thenReturn(...)	Stub a method return
verify(...)	Assert a method was called
AAA	Arrange, Act, Assert structure
Test pyramid	Many unit, some integration, few e2e